Table 6: Serial IMPLY-based implementation of the AND gate.

Step	Operation	Equivalent Logic
1	$S_1 = 0$	FALSE (S_1)
2	$S_2 = 0$	FALSE (S_2)
3	$(a \rightarrow 0) \equiv (a \rightarrow S_1) = S_1{}'$	$S_1{}' = \bar{a}$
4	$(b \rightarrow S_1{}') = S_1{}''$	$S_1{}'' = \overline{a.b}$
5	$(S_1{}'' \rightarrow 0) \equiv (S_1{}'' \rightarrow S_2) = S_2{}'$	$S_2{}' = a.b = \boldsymbol{AND(a,b)}$

Table 7: Serial IMPLY-based implementation of the half adder [28].

Step	Operation	Equivalent Logic
1	$S_1 = 0$	FALSE (S_1)
2	$S_2 = 0$	FALSE (S_2)
3	$(A \rightarrow 0) \equiv (A \rightarrow S_1) = S_1{}'$	$S_1{}' = \bar{A}$
4	$(B \rightarrow 0) \equiv (B \rightarrow S_2) = S_2{}'$	$S_2{}' = \bar{B}$
5	$(S_1{}' \rightarrow S_2{}') = S_2{}''$	$S_2{}'' = A + \bar{B}$
6	$(B \rightarrow S_1{}') = S_1{}''$	$S_1{}'' = \bar{A} + \bar{B}$
7	$(A \rightarrow B) = B'$	$B' = \bar{A} + B$
8	$A = 0$	FALSE (A)
9	$(S_1{}'' \rightarrow 0) \equiv (S_1{}'' \rightarrow A) = A'$	$A' = A \cdot B = \boldsymbol{C_{out}}$
10	$S_1 = 0$	FALSE (S_1)
11	$(S_2{}'' \rightarrow 0) \equiv (S_2{}'' \rightarrow S_1) = S_1{}'$	$S_1{}' = \bar{A} \cdot B$
12	$(B' \rightarrow S_1{}') = S_1{}''$	$S_1{}'' = A \cdot \bar{B} + \bar{A} \cdot B = A \oplus B = \boldsymbol{Sum}$

First, the implementation algorithms of the AND gate, half adder, and full adder cells in [23, 28] are analyzed separately. The IMPLY-based implementation of the AND gate and the half adder cell [28] are written in (1)–(3), respectively.

$$AND(a,b) = a \cdot b \equiv \big(a \rightarrow (b \rightarrow 0)\big) \rightarrow 0 = \overline{(a \rightarrow \bar{b})} \tag{1}$$

$$Sum_{HA} = \alpha \oplus \beta \equiv (\alpha \rightarrow \beta) \rightarrow \overline{(\bar{\alpha} \rightarrow \bar{\beta})} \tag{2}$$

$$Cout_{HA} = \beta \cdot \alpha \equiv \overline{(\beta \rightarrow \bar{\alpha})} \tag{3}$$

The step-by-step IMPLY-based implementation of the AND gate and half adder cell in serial architecture is presented in Tables 6 and 7, respectively. The output of the first four steps, named α here, is inverted in the final step of the AND (a, b) gate's implementation algorithm ($\alpha \rightarrow 0$). By further analysis of the half adder's implementation algorithm introduced in [28], the authors noted that two inputs of the half adder cell (outputs of the two AND gates) are inverted in the third and fourth steps of the algorithm. In short, the α signal of the two AND gates are inverted serially twice: in the last step of the AND gates implementation algorithm and the third and fourth steps of the half adder's algorithm. Hence, it is possible to overlap these steps to reduce the number of computational steps and energy consumption. The proposed PPU_1's output functions are written in (4) and (5).

$$Sum_{PPU1} = ab \oplus cd = \left[\overline{(a \rightarrow \bar{b})} \rightarrow \overline{(c \rightarrow \bar{d})}\right] \rightarrow \overline{\left[(a \rightarrow \bar{b}) \rightarrow (c \rightarrow \bar{d})\right]} \tag{4}$$

$$Cout_{PPU1} = cd \cdot ab \equiv \overline{\left[\overline{(c \to \bar{d})} \to (a \to \bar{b})\right]} \quad (5)$$

Based on these equations, four input and four work memristors are required to implement the proposed IMPLY-based *PPU1* serially in eighteen computational steps. The implementation algorithm of the proposed *PPU1* is presented in Table 8. The number of computational steps of the proposed *PPU1* is reduced by four compared to the classic cell (an improvement of 18%) without extra work memristors.

Table 8: The implementation algorithm of the proposed *PPU1*.

Step	Operation	Equivalent Logic
1	$S_1 = 0$	$FALSE(S_1)$
2	$S_2 = 0$	$FALSE(S_2)$
3	$(b \to 0) \equiv (b \to S_1) = S_1{}'$	$S_1{}' = \bar{b}$
4	$(a \to S_1{}') = S_1{}''$	$S_1{}'' = \bar{a} + \bar{b}$
5	$(d \to 0) \equiv (d \to S_2) = S_2{}'$	$S_2{}' = \bar{d}$
6	$(c \to S_2{}') = S_2{}''$	$S_2{}'' = \bar{c} + \bar{d}$
7	$S_3 = 0$	$FALSE(S_3)$
8	$S_4 = 0$	$FALSE(S_4)$
9	$(S_1{}'' \to 0) \equiv (S_1{}'' \to S_3) = S_3{}'$	$S_3{}' = ab$
10	$(S_2{}'' \to 0) \equiv (S_2{}'' \to S_4) = S_4{}'$	$S_4{}' = cd$
11	$(S_1{}'' \to S_2{}'') = S_2{}'''$	$S_2{}''' = ab + (\bar{c} + \bar{d})$
12	$(S_4{}' \to S_1{}'') = S_1{}'''$	$S_1{}''' = (\bar{a} + \bar{b}) + (\bar{c} + \bar{d})$
13	$(S_3{}' \to S_4{}') = S_4{}''$	$S_4{}'' = (\bar{a} + \bar{b}) + cd$
14	$S_3 = 0$	$FALSE(S_3)$
15	$(S_2{}''' \to 0) \equiv (S_2{}''' \to S_3) = S_3{}'$	$S_3{}' = (\bar{a} + \bar{b}) \cdot cd$
16	$(S_4{}'' \to S_3{}') = S_3{}''$	$S_3{}'' = ab \cdot (\bar{c} + \bar{d}) + (\bar{a} + \bar{b}) \cdot cd = ab \oplus cd = \alpha \oplus \beta = \boldsymbol{Sum}$
17	$S_4 = 0$	$FALSE(S_4)$
18	$(S_1{}''' \to 0) \equiv (S_1{}''' \to S_4) = S_4{}'$	$S_4{}' = ab \cdot cd = \alpha \cdot \beta = \boldsymbol{C_{out}}$

The method of overlapping computational steps is applied for other PPUs of the serial unsigned array multiplier, *PPU2*, and *PPU3* as well. As far as the authors know, the serial IMPLY-based full adder proposed in [23] has the minimum hardware complexity and computational steps compared to previous works. The output functions of this full adder cell are written in (6) and (7), and its implementation algorithm is presented in Table 9.

$$Sum_{FA} = x \oplus y \oplus C_{in} \equiv \left[(\bar{x} \to y) \to \left((x \to \bar{y}) \to C_{in}\right)\right] \to \left(\overline{(x \oplus y)} \to \overline{C_{in}}\right) \quad (6)$$

$$Cout_{FA} = x \cdot y + C_{in}(x + y) \equiv \overline{\left[(\bar{x} \to y) \to \overline{\left((x \to \bar{y}) \to C_{in}\right)}\right]} \quad (7)$$

The IMPLY operation between the input memristors (a_{in} and b_{in}) and two work memristors (S_1 and S_2) which are both initialized to zero (RESET) is executed in the third and fourth computational steps of the serial full adder's implementation algorithm [23]. In these two computational steps, inputs a_{in} and b_{in} are inverted and temporarily stored in work memristors S_1 and S_2, as shown in Table 9.

Table 9: Serial IMPLY-based implementation of the full adder [23].

Step	Operation	Equivalent Logic
1	$S_1 = 0$	$FALSE(S_1)$
2	$S_2 = 0$	$FALSE(S_2)$
3	$(a \to 0) \equiv (a \to S_1) = S_1{}'$	$S_1{}' = \bar{a}$
4	$(b \to 0) \equiv (b \to S_2) = S_2{}'$	$S_2{}' = \bar{b}$
5	$(S_1{}' \to b) = b'$	$b' = a + b$
6	$(a \to S_2{}') = S_2{}''$	$S_2{}'' = \bar{a} + \bar{b}$
7	$a = 0$	$FALSE(a)$
8	$(b' \to 0) \equiv (b' \to a) = a'$	$a' = \bar{a} \cdot \bar{b}$
9	$(S_2{}'' \to a') = a''$	$a'' = ab + \bar{a}\bar{b} = \overline{a \oplus b}$
10	$S_1 = 0$	$FALSE(S_1)$
11	$(C_{in} \to 0) \equiv (C_{in} \to S_1) = S_1{}'$	$S_1{}' = \overline{C_{in}}$
12	$(S_2{}'' \to C_{in}) = C_{in}{}'$	$C_{in}{}' = ab + C_{in}$
13	$(a'' \to S_1{}') = S_1{}''$	$S_1{}'' = (a \oplus b) + \overline{C_{in}}$
14	$a = 0$	$FALSE(a)$
15	$(S_1{}'' \to 0) \equiv (S_1{}'' \to a) = a'$	$a' = \overline{a \oplus b} \cdot C_{in}$
16	$S_2 = 0$	$FALSE(S_2)$
17	$(C_{in}{}' \to 0) \equiv (C_{in}{}' \to S_2) = S_2{}'$	$S_2{}' = (\bar{a} + \bar{b}) \cdot \overline{C_{in}}$
18	$(b' \to S_2{}') = S_2{}''$	$S_2{}'' = \bar{a}\bar{b} + (\bar{a} + \bar{b}) \cdot \overline{C_{in}}$
19	$(b' \to C_{in}{}') = C_{in}{}''$	$C_{in}{}'' = \bar{a}\bar{b} + (ab + C_{in})$
20	$(C_{in}{}'' \to a') = a''$	$a'' = \left[(a + b) \cdot \left((\bar{a} + \bar{b}) \cdot \overline{C_{in}}\right)\right] + \left(\overline{a \oplus b} \cdot C_{in}\right)$ $= (a \oplus b \cdot \overline{C_{in}}) + \left(\overline{a \oplus b} \cdot C_{in}\right) = a \oplus b \oplus C_{in} = \boldsymbol{Sum}$
21	$C_{in} = 0$	$FALSE(C_{in})$
22	$(S_2{}'' \to C_{in}) = C_{in}{}'$	$C_{in}{}' = (a + b) \cdot (ab + C_{in}) = ab + C_{in}(a + b) = \boldsymbol{C_{out}}$

In *PPU₂*, one of the two inputs of the full adder cell is fed by the AND gate, and in *PPU₃*, both of these inputs are fed by the output of the AND gates. As mentioned earlier, α is inverted in the last computational step of the AND gate's implementation algorithm ($\alpha \to 0$). Therefore, it is possible to overlap the last computational step of the AND gate (two AND gates) in *PPU₂* (*PPU₃*) with the third (third and fourth) computational step of the full adder cell implementation algorithm proposed in [23]. As a result, the computational steps of the proposed *PPU₂* cell (25 steps) are improved by 7% compared to the classic design (27 steps). Moreover, the number of computational steps of the proposed *PPU₃* (28 steps) is reduced by 12.5% compared to the classic *PPU₃* (32 steps). This reduction of computational steps is made possible by overlapping repeated serial steps of the subcomponents of these PPUs without increasing the number of work memristors. The output equations of *PPU₂* and *PPU₃* are written in (8)–(11), respectively.

$$\mathbf{PPU_2}: Sum_{FA} = ab \oplus \beta \oplus C_{in} \equiv \left[\left(\left(\overline{(a \to \bar{b})} \to \beta\right) \to \overline{\left((a \to \bar{b}) \to \bar{\beta}\right)}\right) \to C_{in}\right] \to \overline{\left[\left(\left(\left(\overline{(a \to \bar{b})} \to \beta\right) \to \overline{\left((a \to \bar{b}) \to \bar{\beta}\right)}\right) \to \overline{C_{in}}\right)\right]} \quad (8)$$